

ResearchPilot

ANRF PI System

Backend Architecture & Business Logic

Next.js 16 · Prisma ORM · PostgreSQL · NextAuth

Project Overview

What ResearchPilot is and who it serves



Mission

Replace manual spreadsheets & paper forms with a centralised digital system for ANRF-funded research management.



Primary User

Principal Investigators (PIs) at Indian institutions managing ANRF grants.



Roles

PI · Co-PI · JRF · Admin — each with scoped permissions across all modules.



Stack

Next.js 16 App Router, TypeScript, Prisma ORM, PostgreSQL, NextAuth, Tailwind CSS.

Database Schema

PostgreSQL via Prisma ORM — 9 core models

User

id · name · email · role · employeeId · institution

BudgetHead

id · projectId · headName · category · allocatedAmount

Milestone

id · projectId · title · status · dueDate · completedAt

Document

id · projectId · documentType · fileUrl · fileSize · mimeType

Project

id · title · grantType · sanctionNumber · status · totalBudget · pild

Expenditure

id · projectId · budgetHeadId · amount · voucherNumber · expenditureDate

PersonnelRecord

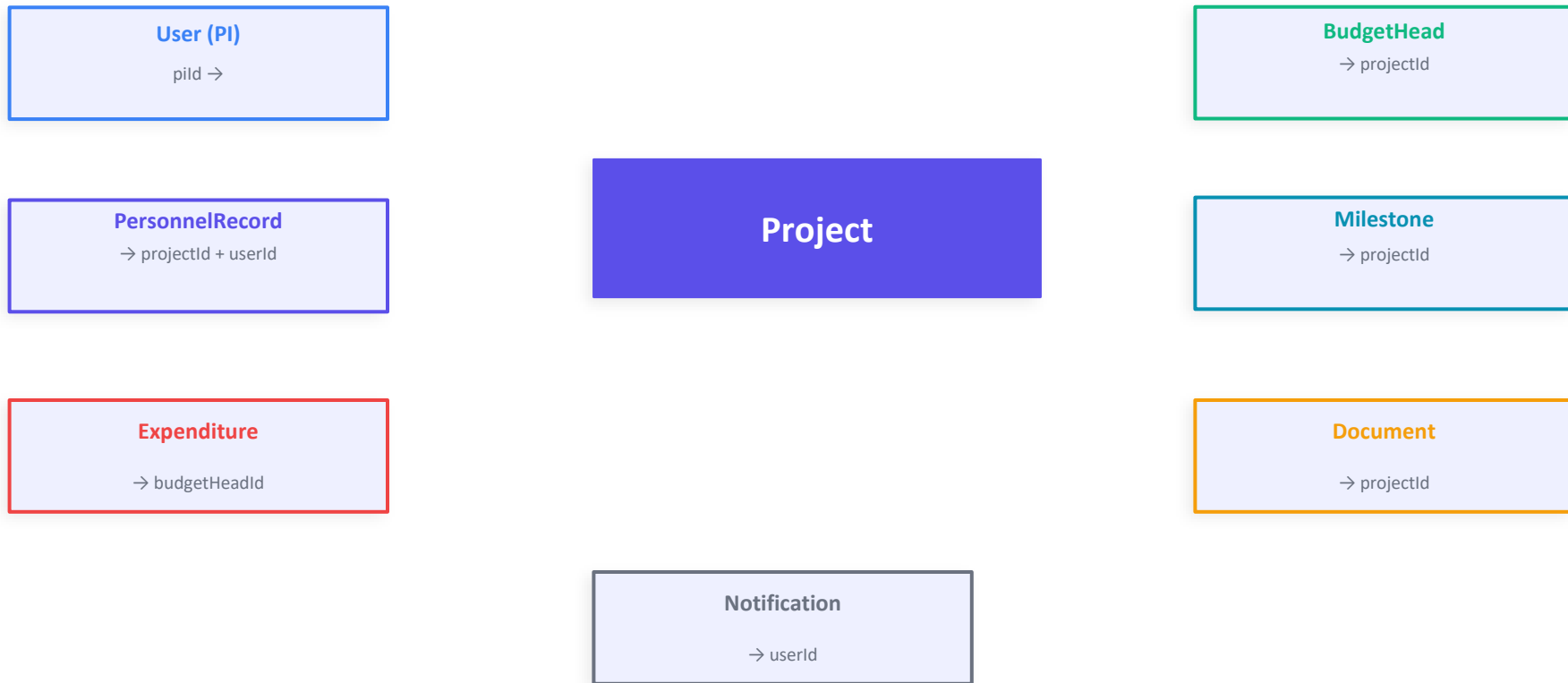
id · projectId · userId · role · stipend · joinDate

Notification

id · userId · title · message · type · read

Entity Relationships

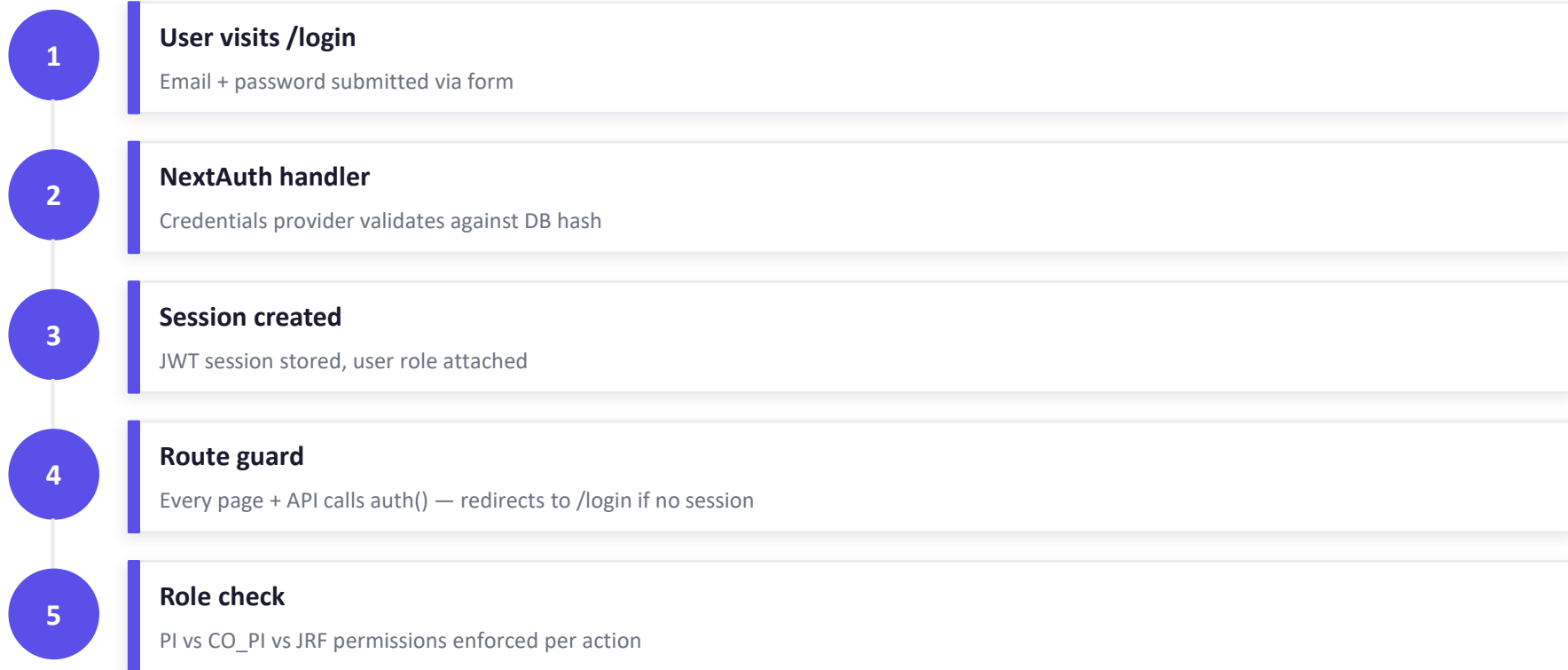
How the core models relate to each other



Cascade delete on all child relations

Authentication System

NextAuth session-based authentication



Role-Based Access Control

Permissions enforced at API and UI layers

Permission	PI	CO-PI	JRF
Create Project	✓	✗	✗
Edit Project	✓	✗	✗
Add Team Member	✓	✗	✗
Add Milestone	✓	✓	✗
Update Milestone	✓	✓	✗
View Financials	✓	✓	✗
Add Expenditure	✓	✗	✗
Add Budget Head	✓	✗	✗
Upload Document	✓	✓	✗
View Documents	✓	✓	✓
View Team	✓	✓	✓

API Routes Architecture

RESTful endpoints built with Next.js App Router

/api/projects	GET · POST	List projects (PI or member), create new (PI only)
/api/projects/[id]	GET · PATCH	Fetch single project, update status/fields
/api/budget-heads	GET · POST · DELETE	Manage budget heads per project
/api/expenditures	GET · POST · DELETE	Record and delete spending per budget head
/api/milestones	GET · POST · PATCH · DELETE	Full CRUD on project milestones
/api/personnel	GET · POST · DELETE	Add/remove team members, inject PI synthetically
/api/documents	GET · POST · DELETE	Upload/list/delete files from disk storage
/api/notifications	GET · PATCH	List notifications, mark read (single or all)

ANRF Business Rules

Domain constraints enforced server-side on every request



Max 8 Active Projects per PI

Checked against ACTIVE + EXTENDED status count before creating any new project. Returns 400 if exceeded.



Max 4 ARG Projects per PI

ARG grant type has a stricter cap. Validated separately on POST /api/projects before insert.



No COMPLETED / UNDER_REVIEW Status

Valid statuses are strictly: ACTIVE, EXTENDED, CLOSED, PENDING. Any other value is rejected at the schema level.



PI-Scoped Data Access

All queries filter by pi.email OR personnelRecords.user.email — users can never access projects they have no relation to.

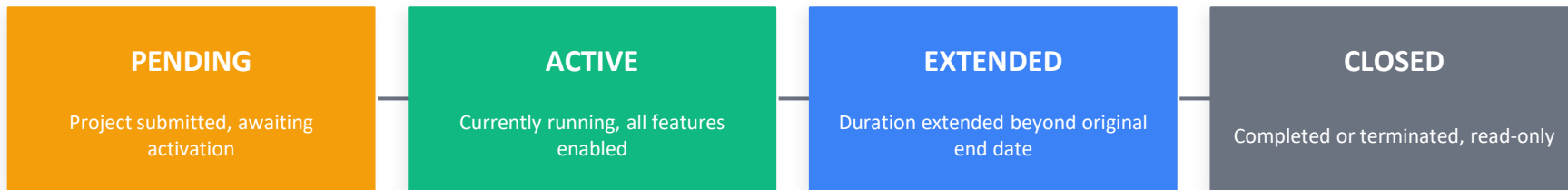


PI-Only Project Creation

CO-PI and JRF roles receive 403 Forbidden on POST /api/projects. Role checked against the User table, not the session claim.

Project Lifecycle

State machine: valid statuses and transitions

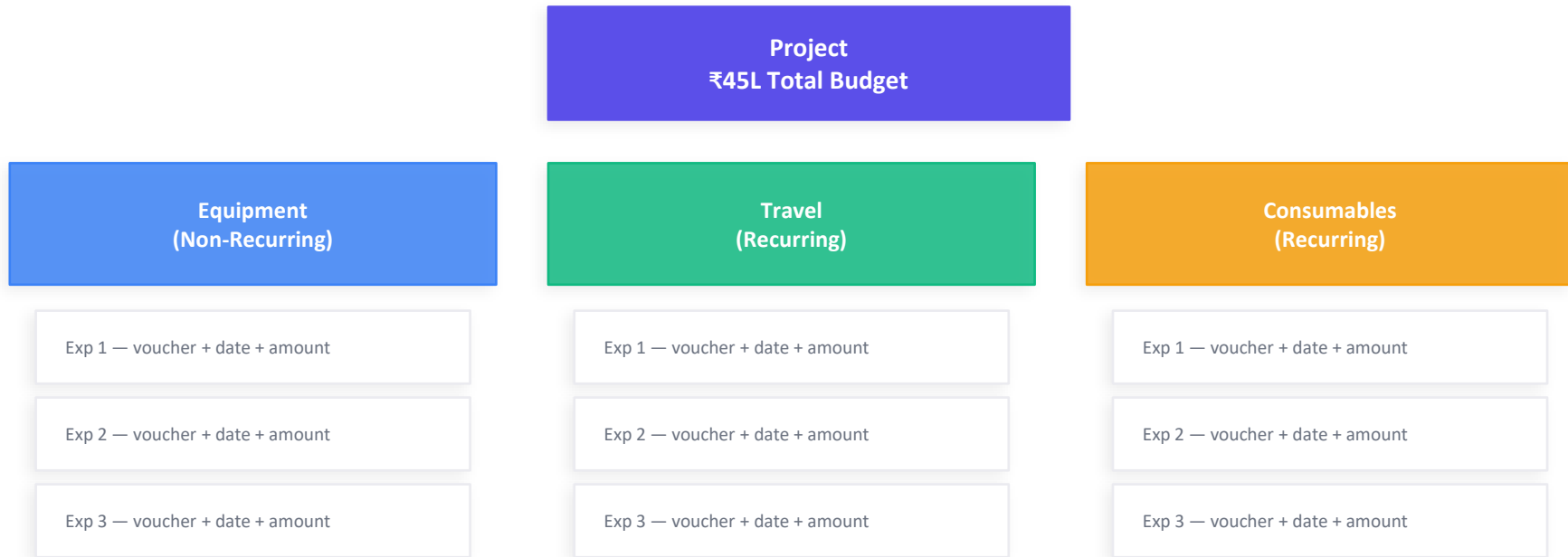


Forbidden values: COMPLETED, UNDER_REVIEW — rejected at API level

- Status is changed via PATCH /api/projects/[id]
- Active count (ACTIVE + EXTENDED) checked against max-8 rule on each new project
- CLOSED projects remain readable — no cascade delete triggered by status change

Budget & Financials Logic

Two-level structure: Budget Heads → Expenditures



Over-budget detection: $spent > allocatedAmount \rightarrow$ UI turns red + shows overflow amount

Personnel & Team Logic

How team membership is stored and the PI injection pattern

PersonnelRecord Model

- `id: String (cuid)`
- `projectId: String → Project`
- `userId: String → User`
- `role: Role enum`
- `joinDate: DateTime`
- `stipend: Decimal?`
- `endDate: DateTime?`
- `unique(projectId, userId)`



Synthetic PI Injection

- Problem: PI is stored as `pild` on `Project`, not in `PersonnelRecord`.
- Solution: `GET /api/personnel` fetches `personnelRecords` AND `project.pi` separately.
- If PI is not already in the records list, they are injected as a synthetic entry with role: `'PI'`.
- Synthetic ID format: `pi-synthetic-{pild}-{projectId}` — unique per project to avoid React key collisions.
- This ensures all users see the PI listed in the team view without schema changes.

Document Management

File upload, storage, and retrieval system



Upload

Client sends multipart form to POST /api/documents. File saved to public/uploads/{userId}/ on local disk.



Notification

On successful upload, a Notification record is auto-created for the PI — type: INFO, with link to documents tab.



Delete

DELETE /api/documents/[id] removes the DB record. File on disk must be cleaned separately in production.



Metadata

Title, documentType (enum), fileUrl, fileSize, mimeType stored in Document table with projectId + uploadedById.



Document Types

PROPOSAL · PROGRESS_REPORT · EXPENDITURE_STATEMENT · UTILIZATION_CERTIFICATE · COMPLETION_REPORT · OTHER



Access Control

PI + CO-PI can upload. All project members can view. Only PI can delete. Role checked from DB user record.

Prisma ORM Deep Dive

Schema design decisions and query patterns

Enums

```
Role, GrantType, ProjectStatus,  
MilestoneStatus, DocumentType,  
BudgetCategory
```

Cascade Delete

```
BudgetHead → Expenditure  
Project → Milestone, Document,  
PersonnelRecord, Expenditure  
User → Notification
```

Decimal Type

```
totalBudget, allocatedAmount,  
stipend → Prisma Decimal  
Must convert: Number(x)  
before passing to client
```

Unique Constraints

```
Project: sanctionNumber  
PersonnelRecord:  
unique(projectId, userId)
```

Deployment & Infrastructure

Current setup and production roadmap

Current State

- Local: localhost:3000
- Network: 172.16.34.x:3000
- DB: PostgreSQL (local)
- Files: public/uploads/ on disk
- Auth: NextAuth JWT sessions
- Dev server: Turbopack (next dev)
- No CI/CD pipeline
- No automated tests

Production Roadmap

- Deploy to Vercel / Railway
- PostgreSQL → Neon or Supabase
- File storage → AWS S3 / Cloudflare R2
- Environment secrets via .env.production
- Add Jest + Playwright tests
- GitHub Actions CI/CD
- Rate limiting on API routes
- Audit log for expenditures